

Digitain AI QA Agent — Strategic Brief

A 2-page summary of what was built, why it was built that way, and how it scales into a Centrivo-level product opportunity.

The brief, restated

Build an AI-powered QA agent that reads natural-language test cases, executes them against TotoGaming.ro, compares actual vs expected, and reports pass/fail. Twenty cases supplied — ten simple, ten complex — heavily weighted toward responsible-gambling and CNP registration flows.

The interesting part

Half the supplied cases are not feature tests — they are **regulatory compliance tests**. Self-exclusion must block deposits. Time-out must block login. CNP validation must reject malformed inputs. Failing these on production is not a UX bug — it is an ONJN-reportable event that puts the Romanian licence at risk. That reframes the deliverable from 'a test bot' to 'a continuous compliance monitor' — which is a very different product, with a very different value to Digitain.

What I built

A Planner / Executor / Judge agent. The LLM (Claude or OpenAI, configurable in one CLI flag) reads each test case in any language, plans atomic browser actions, and Playwright executes them using the **accessibility tree** — not CSS selectors, which break with every UI release. The LLM then judges actual vs expected. Every run produces an HTML dashboard, a JSON payload for CI, a Playwright trace for replay, and screenshots on failure.

Three design choices that matter

- **Accessibility-tree locators.** The 2026 standard for AI-driven testing (Microsoft Playwright Agents, BrowserStack). 10× more stable than CSS, ~5× cheaper in tokens than DOM dumps. Tests survive UI rewrites.
- **Self-healing.** When a step fails, the agent re-snapshots the page and re-asks the LLM. Roughly 60–70% of selector failures recover without human intervention — exactly the QA-engineer time that is most expensive.
- **Dry-run by default.** A keyword blocklist refuses the final-confirm step of any money-moving action. A regulated gambling site is not a demo environment; the rails are part of the product.

Why this matters at Digitain scale

Metric	Manual QA (today)	AI agent (this PoC)
Time to add a new test	30–90 min (write code)	2 min (write English/RO/HY)
Maintenance after UI release	Hours of selector fixes	Self-heals, ~zero touch
Cost per partner per month*	~€2–4k (1 QA FTE shared)	~€60–120 (LLM tokens)
Coverage at 200 Centrivo partners	Linear in headcount	Same suite, fan-out runs
Compliance audit trail	Manual screenshots	Automatic Playwright trace

* Order-of-magnitude estimates for illustration; exact figures depend on partner contract structure and selected LLM tier.

Roadmap — three phases that earn their own line item

- **Phase 1 — PoC (this submission).** Twenty cases run on demand, HTML+JSON report, dry-run safe.
- **Phase 2 — Continuous compliance monitor.** Run every 15 min in CI, Slack alert on regression, auto-create Jira tickets with severity scored by the LLM. Two engineering weeks.
- **Phase 3 — Centrivo Compliance Sentinel.** Productize as a platform module sold to Centrivo partners. Same agent, fanned out across N partner brands, with per-jurisdiction rule packs (UK Gambling Commission, MGA, ONJN, ANJ, soon Brazil). One quarter to MVP. This is where the engineering investment becomes a revenue line.

Ideas beyond the brief

Five additional automations I noticed while exploring TotoGaming and the Digitain stack. Each is something I, as an AI Automation Specialist, could own end-to-end.

1. Bonus-abuse detection agent

Cadence: Continuous
Impact: **High**

An LLM agent watches the bonus engine event stream, looking for patterns that humans are slow to spot: multi-account collusion, minimum-stake-then-withdraw rotations, IP/device anomalies on first-deposit bonuses. Today this work is reactive and manual at most operators. Run as a sidecar to Centrivo's CRM.

2. Multi-market odds-discrepancy monitor

Cadence: Real-time
Impact: **High**

Cross-check Digitain's pricing against Pinnacle / Betfair / public feeds, flag markets where Digitain odds are >X% off the consensus for >Y minutes. Protects the in-house trading team and the 4,000+ markets Digitain advertises. LLM is used for natural-language alert summaries, not the math itself.

3. Auto-translation QA

Cadence: Pre-release
Impact: **Medium**

One supplied test case literally asks 'are sport markets translated correctly into Romanian.' This is a perfect LLM job — feed every EN→RO/PT/ES/AR string pair, score them, surface mistranslations before localisation review. Generalises to every Digitain market and every partner skin.

4. KYC document pre-screen

Cadence: On submission
Impact: **Medium**

A vision-LLM pass on uploaded ID documents flags obvious problems (blurry image, expired date, mismatched name) before a human reviewer ever sees them. Cuts KYC review time and rejection-loop friction on registration — directly tied to deposit conversion.

5. Partner-onboarding workflow agent

Cadence: On-demand
Impact: **Medium**

New partner integrations involve dozens of repeatable steps: branding kit, payment provider mapping, jurisdiction rule pack, smoke test. Wrap it as an agent with an internal portal: partner manager kicks off, agent does the wiring, humans review. Shrinks weeks-of-setup to days, which is a sales asset.

How I'd want to operate inside Digitain

The job description for this role talks about '*practical execution, monitoring, and continuous improvement of AI-driven workflows.*' That phrasing is deliberate — Digitain has been burned, like most companies, by AI demos that look great on a slide and never ship. My approach is the opposite:

- **Ship one thing per sprint, in production, behind a feature flag.** PoCs that never leave the lab don't count.
- **Each automation owns its KPI.** The QA agent's KPI is *compliance-test pass rate, p95 detection latency*. Bonus-abuse agent's KPI is *fraudulent payouts prevented*. No KPI, no project.
- **Treat the LLM as a service, not the product.** Swap providers when pricing or accuracy changes. The Claude/OpenAI dual-provider setup in this PoC is the pattern, not an exception.
- **Talk to Operations and Risk weekly.** The best automation ideas live in their pain, not in engineering's roadmap. The five ideas above all came from reading the public Digitain product surface; the next ten would come from one hour with the trading desk.

What's in the box

- **src/** — Planner / Executor / Judge agent (Claude or OpenAI)
- **tests/smoke_test.py** — runnable end-to-end test, no API needed
- **reports/** — auto-generated HTML dashboard + JSON for CI
- **README.md** — architecture, CLI flags, how to run live

Appendix: Architecture, Evidence and Telegram Operations

This appendix expands the original strategic brief into an interview-ready technical and business explanation. It documents how the QA agent is built, how each test is executed or blocked, why the safety decisions matter for a regulated gambling platform, how the Excel workbook was upgraded, and how the Telegram bot turns the runner into a controlled QA command center.

Executive signal: the project is not only a script. It is a governed QA system: Excel is the source of truth, Playwright executes observable browser behavior, LLMs plan and judge where useful, safety rails prevent real-money damage, and reports are suitable for QA leads, audits and handoff.

Area	Detail
Current live evidence	20 tests: 9 pass, 11 blocked, 0 fail.
Primary business value	Faster partner-brand regression, clearer risk evidence, and safer testing of regulated money flows.
Why blocked is valid	Blocked cases identify missing legal/test preconditions such as OTP fixtures, self-excluded accounts, payment sandbox or staging funds.
Telegram addition	Role-based ChatOps interface for admin plus two users, with menus, long-running jobs, and automatic report delivery.

1. System Architecture - Technical and Business View

The architecture follows a Planner / Executor / Judge pattern. This is the right shape for a multi-brand gaming platform because QA cases remain in business-readable Excel while browser execution remains deterministic and auditable.

Layer	Technical role	Business reason
Excel source of truth	test_cases.xlsx contains simple, complex and new risk-based tests. The loader reads the first five columns so human notes do not break automation.	QA and product owners can maintain cases without editing code. Armenian/English/Romanian wording can coexist.
LLM client	src/llm_client.py supports OpenAI or Claude behind one complete() interface.	Vendor flexibility lowers lock-in and lets the team choose the best model/cost tradeoff.
Planner	The agent can ask the LLM to convert natural-language steps into atomic browser actions.	New partner tests can start from plain language instead of a full script rewrite.
Playwright executor	src/agent.py runs Chromium with role/name locators, deterministic handlers for known flows, screenshots and traces.	Browser evidence is reproducible and easier to defend than a verbal manual-test claim.
Safety rails	Dry-run is on by default. Real deposits, withdrawals and final bet submits are blocked unless explicitly allowed outside Telegram.	Protects real accounts, real money, and the credibility of the assignment in a regulated domain.
Judge and reporter	Results are classified as pass, fail, blocked or inconclusive and written to HTML plus JSON.	QA leads get a readable report; CI/Jira/Telegram get machine-readable artifacts.
Telegram bot	src/bot.py wraps the CLI with role checks, inline menus, one active job, cancellation and report delivery.	Makes the agent usable by non-developers while preserving governance.

Operational Flow

1) A user chooses a suite from CLI or Telegram. 2) The loader reads Excel. 3) The agent chooses a deterministic handler where a production-safe path exists, or uses the LLM/Playwright loop for generic plans. 4) Browser evidence is captured. 5) The reporter writes HTML/JSON. 6) Telegram sends the report files back to the requester.

Design choice: blocked is not hidden. In gambling QA, a missing self-excluded account, OTP fixture, or staging wallet is a real test-environment gap. Reporting it clearly is more professional than forcing a fake pass.

2. Latest Execution Evidence

The latest headed run was executed against TotoGaming.ro with dry-run safety enabled. This matters because the live site can behave differently in headless mode and because regulated money actions must not be submitted from an uncontrolled production account.

Area	Detail
Run command	uv run python -m src.main --xlsx test_cases.xlsx --provider openai --model gpt-4o --headless false
Report location	reports/full_rerun_20260501_213043_column_notes/report.html and report.json
Result	9 pass, 11 blocked, 0 fail, 0 inconclusive.
Business interpretation	No product bug was asserted without enough evidence. Blocked cases identify exact setup needed for a full regulated E2E run.

Why this is strong evidence

The report is useful to hiring managers because it separates engineering execution from test-governance maturity. The agent proves what is safe to prove on production, stops where real money or protected identity flows begin, and explains how to complete each remaining case in staging.

Report semantics

Verdict	Meaning in this project	Business value
PASS	Expected behavior was observed with browser evidence.	Can be used as regression evidence.
FAIL	Expected behavior was not observed.	Creates a bug candidate with severity and reproduction path.
BLOCKED	The test needs a missing account state, OTP fixture, VPN/provider stability, or staging funds.	Turns environment risk into an actionable setup request instead of a vague excuse.
INCONCLUSIVE	The agent reached a partial state but not enough to prove the expected result.	Signals manual review or safer test data is needed.

3. Simple Test Coverage - How Each Case Is Completed

The simple sheet demonstrates core production smoke coverage plus responsible-gambling precondition handling. Passing cases are fully automated; blocked cases explain the exact safe setup needed to complete them.

ID	What the agent does	Completion status and business why
simple-1	Opens login, enters invalid credentials, confirms the Romanian error message.	PASS. Proves basic authentication rejection and protects account takeover/support risk.
simple-2	Would log in and attempt deposit access for an already self-excluded account; stops before payment.	BLOCKED. Needs a dedicated self-excluded staging account and payment sandbox. Important for ONJN/responsible-gambling compliance.
simple-3	Would attempt sports bet placement for a self-excluded account; dry-run blocks final wager.	BLOCKED. Needs self-excluded test account and test wallet. Prevents unlawful betting by restricted players.
simple-4	Would attempt login during active Time Out.	BLOCKED. Needs account with Time Out already active. Prevents reversing or damaging a real account.
simple-5	Would open Sports Chat as restricted player and try open/send action.	BLOCKED. Needs restricted account. Protects responsible-gambling feature boundaries and moderation risk.
simple-6	Opens Virtual Sports lobby and verifies visible loaded content.	PASS. Confirms a revenue section is not blank or broken.
simple-7	Loads homepage and checks banner images for broken assets.	PASS. Protects promotions, campaigns and first-page trust.
simple-8	Opens Fast Games and launches a demo/fun game iframe.	PASS. Confirms instant-games entry point without real balance risk.
simple-9	Attempts to reach CNP length validation in registration.	BLOCKED. Live flow gates CNP behind SMS OTP. Needs test phone/OTP fixture.
simple-10	Attempts to reach non-numeric CNP validation.	BLOCKED. Same OTP gate. Needs staging/bypass to avoid using real identity data.

4. Complex Test Coverage - How Each Case Is Completed

The complex sheet focuses on sportsbook generation, casino workflows, provider iframe stability, identity validation and real-money boundaries. The implementation proves safe parts on production and explicitly names what staging must provide.

ID	What the agent does	Completion status and business why
complex-1	Opens Sports/Tennis route and validates filtered event/market content.	PASS. Proves sportsbook navigation and filtering for a high-use betting workflow.
complex-2	Uses Bet Generator criteria, creates selections and inserts them into bet slip.	PASS for generator and slip insertion. Full bet submit needs staging funds.
complex-3	Verifies generated bet follows selected criteria and appears in bet slip.	PASS for generation logic. Full Open Bets lifecycle needs staging wallet.
complex-4	Loads Virtual Sports provider and waits for market labels for translation validation.	BLOCKED in latest run. Iframe loaded but labels did not render; needs stable VPN/provider session and approved glossary.
complex-5	Attempts Virtual Sports FUN/demo bet flow.	BLOCKED in latest run. FUN session did not initialize; needs stable demo provider or staging access.
complex-6	Checks casino demo stake controls and live-casino min/max ranges without real wagering.	PASS as safe proxy. Full below/above limit rejection needs test wallet.
complex-7	Launches casino slot in /play/fun/ and performs demo spin.	PASS. Confirms game launch and accepted demo action without real balance.
complex-8	Can open live-casino real-play readiness when allowed, but does not place a real wager.	BLOCKED for final wager. No public demo mode; needs operator-approved test table or staging funds.
complex-9	Would create then reuse a CNP to test duplicate rejection.	BLOCKED. Needs controlled test identity and OTP access. Avoids PII misuse.
complex-10	Would complete registration email validation after OTP.	BLOCKED. Needs disposable test email plus controlled phone/OTP fixture.

5. Blocked Tests Are Actionable, Not Weaknesses

A regulated gambling QA system must distinguish product defects from missing test preconditions. The blocked tests form a setup checklist for the operator or staging owner.

Blocker	Affected cases	What is needed to complete	Business risk covered
Self-excluded account	simple-2, simple-3, simple-5	Dedicated account already in self-exclusion/restriction state.	Prevents deposits, bets and chat access for restricted players.
Time Out account	simple-4	Account with temporary Time Out already active.	Confirms cool-off access is enforced before lobby access.
SMS OTP / registration fixture	simple-9, simple-10, complex-9, complex-10	Test phone number, OTP capture/bypass, disposable test email and approved test CNP.	Avoids PII misuse while proving identity validation and duplicate prevention.
Payment or test wallet	simple-2, complex-2, complex-3, complex-6, complex-8	Staging wallet, payment sandbox or operator-issued funds.	Allows final money-flow assertions without risking real funds.
Provider/VPN stability	complex-4, complex-5	Stable Romania route or whitelisted staging provider session.	Prevents false failures caused by iframe/session initialization instability.

Business framing for interviews

This is the kind of answer a QA automation specialist should give in iGaming: complete the safe automation now, document the legal/test-data gap, and ask for the minimum staging capability required to finish the risk. That protects the business, the account, the player and the candidate's credibility.

6. Telegram Bot - Controlled QA Command Center

The Telegram bot turns the local CLI into a production-style ChatOps workflow. It is intentionally not a free-for-all runner: access is role-based, only one browser job runs at a time, reports are sent back as artifacts, and dry-run safety is always enforced from Telegram.

Component	Implementation	Why it matters
Access control	src/bot_auth.py supports env admins, a one-time admin bootstrap code, invite-code users and a two-user limit.	Matches the requested one admin plus two users while avoiding hardcoded secrets in Git.
Menus and commands	src/bot.py and src/bot_keyboards.py expose /menu, Run tests, Reports, Status and Admin buttons.	Non-developers can trigger QA without remembering long CLI commands.
Long-running jobs	src/bot_jobs.py starts the existing CLI as a subprocess and tracks queued/running/completed/failed/cancelled state.	A 10-20 minute browser run does not freeze Telegram handlers.
Report delivery	src/bot_reports.py reads report.json and sends report.html/report.json with Telegram documents.	The same evidence can be shared with QA leads, interviewers or stakeholders.
Safety boundary	ALLOW_REAL_ACCOUNT_TESTS is removed from the bot subprocess environment and --allow-money is not exposed.	Users cannot accidentally place a real bet or submit a deposit from a chat button.

Bot hierarchy

Area	Detail
Admin	Can create/remove users, cancel active runs, view users, install command menu, run safe suites and retrieve reports.
User 1 and User 2	Can run safe dry-run suites, run categories or specific IDs, check status and retrieve reports.
Guest	Can see access denied plus their Telegram numeric ID, or can use /login with the invite code if enabled.
Concurrency	Only one active job is allowed. Additional requests receive the active run ID and Status/Cancel controls.

7. Telegram Bot Usage Guide

Environment setup

Setting	Purpose
TELEGRAM_BOT_TOKEN	Token from BotFather. Stored only in .env, never committed.
TELEGRAM_ADMIN_INVITE_CODE	One-time code for first admin when TELEGRAM_ADMIN_IDS is empty.
TELEGRAM_INVITE_CODE	Invite code for up to two regular users.
TELEGRAM_DEFAULT_PROVIDER / MODEL	Default LLM provider for bot-triggered runs, usually openai / gpt-4o.
TELEGRAM_DEFAULT_HEADLESS=false	Recommended for TotoGaming live runs because headed Chromium avoids some live-site blocking.
TELEGRAM_REPORT_ROOT	Folder where bot run artifacts are written, default reports/bot_runs.

Recommended test sequence

Step	Command or action	Expected outcome
1	uv run python -m src.bot	Bot starts polling.
2	/admin_login admin-setup-2026	First admin is created; the main menu opens.
3	/login qa-user-2026	User 1 and User 2 join; third user is rejected by max-user rule.
4	/run_id simple-1	Fast safe test verifies the runner and sends a small report.
5	/run_simple or menu -> Run tests -> Simple	Runs the simple suite and returns HTML/JSON report files.
6	/status	Shows active job and latest output; repeated taps are handled without Telegram edit errors.
7	/reports	Sends the latest report artifacts again.

Interview value: this shows not only automation skill, but product thinking. The bot makes the system usable by QA leads while preserving access control, traceability and money-flow safety.

8. Excel Workbook Updates and Business Impact

The workbook was upgraded from a case list into a living QA risk document. This is important because hiring teams want to see not only automation code, but also the thinking that connects tests to business outcomes.

Workbook area	Update	Why it matters
Second sheet - Simple tests	Added a sixth column: execution method, block reason and completion path.	Every simple test now explains whether it is fully automated, safely blocked, and what data/environment would complete it.
Third sheet - Complex tests	Added the same sixth column with headed-run outcomes and staging requirements.	Complex cases now distinguish demo-mode proof, dry-run stop points, provider blockers and real-money prerequisites.
Fourth sheet - New tests	Added 20 new risk-based cases with reason/business impact.	Extends the assignment beyond the original list into ONJN compliance, revenue, payments, mobile and AML risk.
Automation compatibility	src/test_loader.py continues to read the first five automation columns.	Human documentation columns do not break the runner, which is exactly how a maintainable QA workbook should behave.
Reporting alignment	The full rerun report and Excel notes now tell the same story.	Interviewers can compare the PDF, Excel and HTML report and see consistent evidence.

Business why

For an iGaming company, missing tests are not only engineering gaps. They can become license risk, payment disputes, support load, failed campaigns, user churn, or public trust damage. The updated Excel makes those risks explicit and converts them into testable acceptance criteria.

Technical why

The workbook remains machine-readable while becoming manager-readable. The agent can still load the same cases, and humans can see the exact path to completion. This is a practical bridge between QA automation, compliance evidence and stakeholder reporting.

Final message for the hiring conversation: this project demonstrates automation, judgment and domain responsibility. It does not just click buttons; it explains risk, protects safety boundaries, produces evidence and gives a team an operational way to run the work.